AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2025

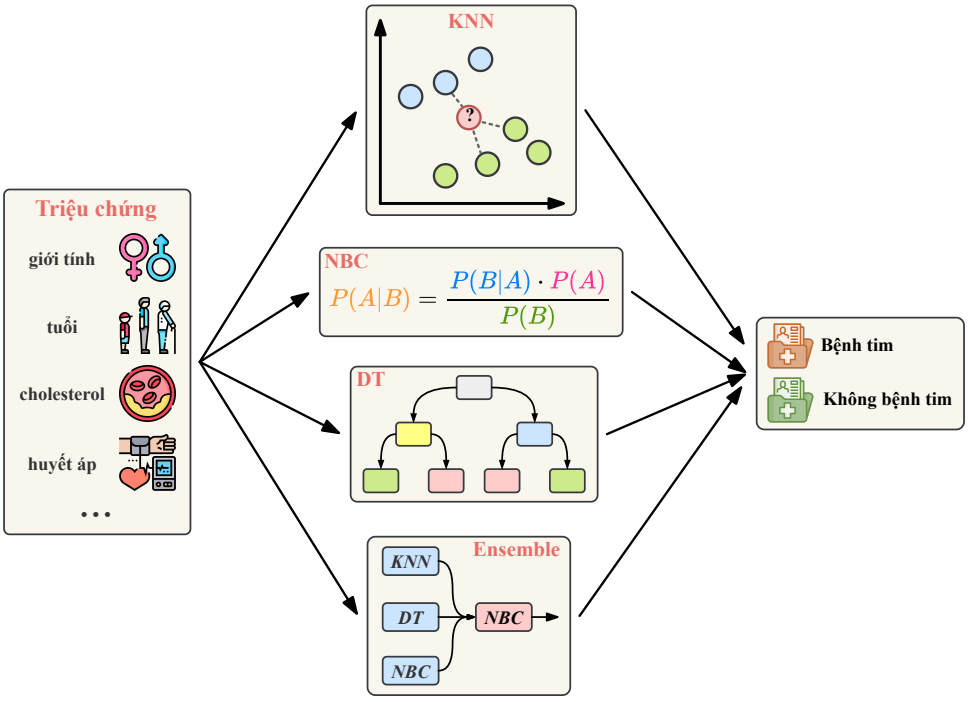
Project: Heart Disease Diagnosis

Dương Đình Thắng Nguyễn Anh Khôi Đinh Quang Vinh

I. Dẫn nhập

Bệnh tim mạch hiện là nguyên nhân gây tử vong hàng đầu trên thế giới, với khoảng 20,5 triệu ca vào năm 2023, chiếm xấp xỉ 32% tổng số ca tử vong toàn cầu [1]–[3]. Tại Việt Nam, ước tính của WHO năm 2016 cho thấy 31% trường hợp tử vong có liên quan đến bệnh tim mạch. Dự báo mới nhất cũng chỉ ra rằng 80% gánh nặng tử vong do bệnh tim sẽ rơi vào các quốc gia có thu nhập từ thấp đến trung bình - nơi việc tầm soát và điều trị còn gặp nhiều hạn chế về chi phí và nguồn lực [3], [4].

Trước thực trạng đó, các giải pháp hỗ trợ chẩn đoán sớm và chính xác đóng vai trò vô cùng quan trọng. Trong những năm gần đây, các mô hình học máy (machine learning) đã nổi lên như một hướng tiếp cận hiệu quả, nhờ khả năng xử lý khối lượng dữ liệu lớn và phát hiện các mối quan hệ ẩn giữa các đặc trưng y tế. Đặc biệt, nhiều nghiên cứu đã chứng minh rằng các thuật toán học máy truyền thống như **Naive Bayes**, **K-Nearest Neighbors**, **Decision Tree**... vẫn được ưa chuộng nhờ tính đơn giản, khả năng diễn giải, và độ chính xác chấp nhận được trong thực tế.



Hình 1: Tổng quát dự án dự đoán bệnh tim bằng các phương pháp học máy.

Mục lục

I. Dẫn nhập **1**

II. Cài đặt chương trình **4**

II.1. Tổng quan về bộ dữ liệu 4

II.2. Kỹ thuật xử lý đặc trưng 6

II.3. Naive Bayes Classifier 14

II.4. K-Nearest Neighbors 18

II.5. Decision Tree 21

II.6. K-means 23

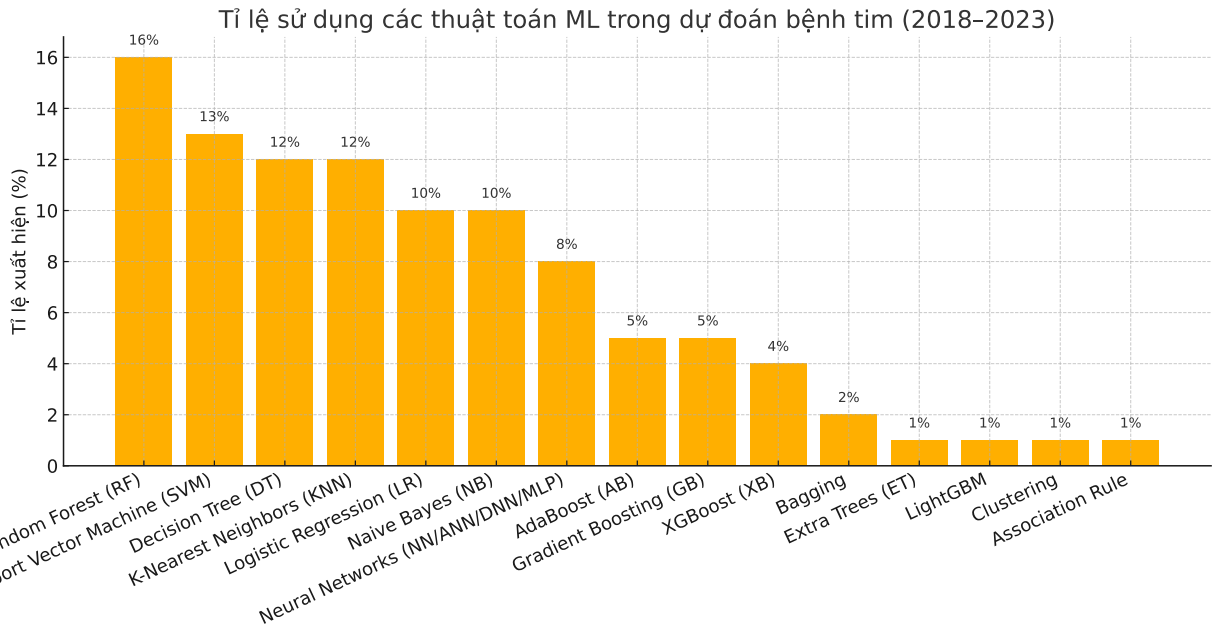
II.7. Cải tiến: Ensemble 25

III. Câu hỏi trắc nghiệm **29**

IV. Tài liệu tham khảo **31**

Phụ lục **32**

Theo một công trình khảo sát [5] (2025) về bài toán liên quan cung cấp góc nhìn về mức độ phổ biến của các kỹ thuật học máy truyền thống thường được sử dụng (trực quan như hình Hình 2 bên dưới). Một cách bất ngờ rằng các mô hình cơ bản như **K-Nearest Neighbors KNN**, **Decision Tree DT** với 12% và **Naive Bayes NB** là 10%, cho thấy các mô hình này vẫn được ưu chuộng vì tính đơn giản mà vẫn đem lại độ chính xác chấp nhận được.



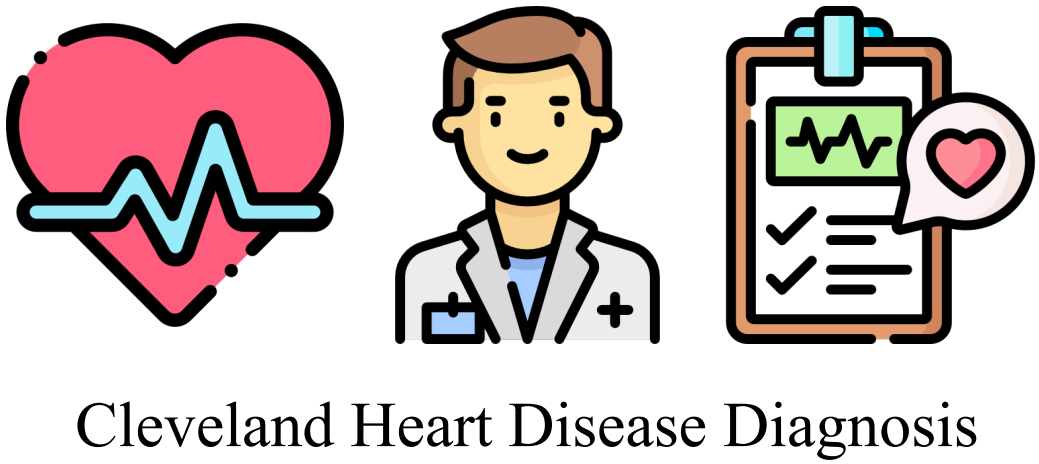
Hình 2: Trực quan tần suất xuất hiện của các mô hình học máy truyền thống trong các công trình nghiên cứu về dự đoán bệnh tim từ năm 2018 đến 2023.

Xuất phát từ bối cảnh này, tài liệu sẽ hướng dẫn cách xây dựng và so sánh nhiều mô hình học máy trên bộ dữ liệu bệnh tim. Nội dung tiếp theo bao gồm: Tổng quan dữ liệu, kỹ thuật xử lý đặc trưng, cùng phần giới thiệu và thực hành với các mô hình học máy trên hai phiên bản dữ liệu: bộ dữ liệu gốc và bộ dữ liệu đã được xử lý đặc trưng.

II. Cài đặt chương trình

II.1. Tổng quan về bộ dữ liệu

"Biết người biết ta, trăm trận trăm thắng", trong học máy, việc hiểu rõ dữ liệu chính là yếu tố quyết định để xây dựng mô hình hiệu quả. Trong dự án này, chúng ta sử dụng bộ dữ liệu Cleveland Heart Disease, một trong bốn tập con thuộc bộ dữ liệu bệnh tim tổng hợp do UCI Machine Learning Repository cung cấp [6]. So với các tập con khác (Hungary, Switzerland, Long Beach VA), Cleveland được lựa chọn rộng rãi trong nghiên cứu nhờ có ít giá trị thiếu, nhãn phân loại rõ ràng và xuất phát từ nghiên cứu y học kinh điển của Detrano và cộng sự [7].



Hình 3: Bộ dữ liệu dự đoán bệnh tim: Cleveland Heart Disease Diagnosis.

Bộ dữ liệu gồm 303 bệnh nhân với 14 đặc trưng y tế phản ánh tình trạng sức khỏe tim mạch, bao gồm các thông tin nhân khẩu học (tuổi, giới tính), các chỉ số lâm sàng (huyết áp, cholesterol, nhịp tim tối đa), các kết quả kiểm tra chuyên sâu (điện tâm đồ, mức độ trầm ST, số mạch máu chính) và thông tin về bệnh lý thalassemia. Mỗi đặc trưng đã được mã hóa dạng số để thuận tiện cho việc huấn luyện mô hình. Bảng dưới đây minh họa một vài mẫu dữ liệu được lấy từ bộ dữ liệu gốc:

Bảng 1: Ví dụ một số mẫu dữ liệu từ bộ Cleveland Heart Disease.

age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0
67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	1
67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0
41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0

Việc nắm vững cấu trúc và ý nghĩa của các đặc trưng này không chỉ giúp lựa chọn mô hình phù hợp mà còn đóng vai trò quan trọng trong các bước xử lý đặc trưng và phân tích kết quả. Ở phần tiếp theo, chúng ta sẽ tìm hiểu các kỹ thuật xử lý đặc trưng cần thiết trước khi huấn luyện mô hình.

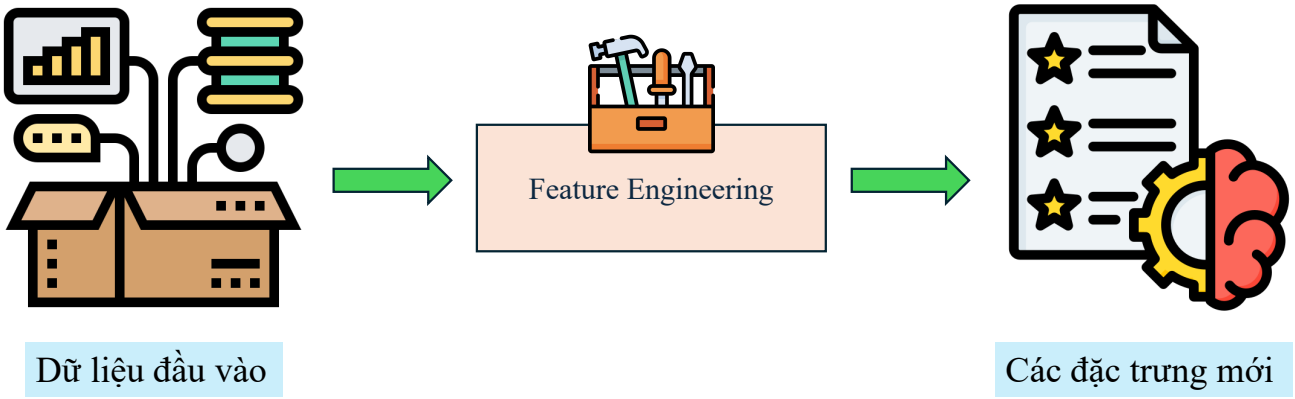
Bảng mô tả đặc trưng bộ dữ liệu Cleveland Heart Disease

Đặc trưng	Mô tả và giá trị mã hóa
`age`	**Tuổi của bệnh nhân** (năm).
`sex`	**Giới tính**. (1 = nam, 0 = nữ)
`cp`	**Loại đau ngực**. (1 = đau thắt ngực điển hình, 2 = không điển hình, 3 = đau không do tim, 4 = không triệu chứng)
`trestbps`	**Huyết áp tâm thu lúc nghỉ** (mmHg).
`chol`	**Nồng độ cholesterol huyết thanh** (mg/dL).
`fbs`	**Đường huyết lúc đói > 120 mg/dL**. (1 = đúng, 0 = sai)
`restecg`	**Điện tâm đồ lúc nghỉ**. (0 = bình thường, 1 = bất thường ST-T, 2 = phì đại thất trái)
`thalach`	**Nhịp tim tối đa đạt được** (lần/phút).
`exang`	**Đau ngực khi gắng sức**. (1 = có, 0 = không)
`oldpeak`	**Mức độ trầm ST do gắng sức so với nghỉ**.
`slope`	**Độ dốc đoạn ST**. (1 = dốc lên, 2 = bằng phẳng, 3 = dốc xuống)
`ca`	**Số mạch máu chính được nhuộm màu** (0–3).
`thal`	**Thalassemia**. (3 = bình thường, 6 = tổn thương cố định, 7 = tổn thương có thể đảo ngược)
`num`	**Nhãn mục tiêu**. (0 = không bệnh, 1–4 = có bệnh)

Các giá trị rời rạc được mã hóa trong ngoặc để phục vụ huấn luyện mô hình.

II.2. Kỹ thuật xử lý đặc trưng

Feature engineering là một kỹ thuật đặc biệt có thể biến đổi và bổ sung đặc trưng từ dữ liệu gốc để mô hình có khả năng học tốt hơn. Việc này giúp làm nổi bật các tín hiệu hữu ích, giảm nhiễu, đồng thời tạo ra các đặc trưng mới có giá trị phân tách cao hơn cho quá trình huấn luyện. Trong bài toán dự đoán bệnh tim, chúng ta có thể tiến hành xử lý đặc trưng nhằm cải thiện chất lượng dữ liệu đầu vào, từ đó nâng cao hiệu quả của các mô hình học máy. Việc so sánh kết quả trên tập dữ liệu gốc và tập đã xử lý đặc trưng sẽ cho thấy mức độ cải thiện rõ rệt của kỹ thuật này. **Lưu ý:** Các bạn có thể tải trước bộ dữ liệu sau khi feature engineering tại phần IV.



Hình 4: Minh họa về kỹ thuật feature engineering khi áp dụng vào một bộ dữ liệu nhằm tạo thêm các đặc trưng mới tốt hơn.

Trước tiên, nạp các thư viện xử lý dữ liệu, tiền xử lý, chọn đặc trưng và trực quan, ngoài ra đặt seed để kết quả có thể lặp lại ổn định.

```
import os
import json
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import VarianceThreshold, mutual_info_classif
from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler,
    FunctionTransformer, MinMaxScaler
)

os.environ["PYTHONHASHSEED"] = "42"
np.random.seed(42)
```

```
random.seed(42)
print("Seed: 42")
```

Tiếp theo là tải tệp dữ liệu Cleveland từ Google Drive bằng `gdown` để chuẩn bị cho các bước làm sạch và tách thành các tập con, các bạn có thể kiểm tra bộ dữ liệu tại phần IV.

```
!gdown 16HPyuXWXPptt5g3xvS_kR_wXAfjpR1Ju
```

Kế tiếp, đặt tên cột theo các kiểu chuẩn, ép kiểu số cho các trường quan trọng, nhị phân hóa cho cột target (0: không bệnh và >1: có bệnh).

```
DATA_PATH = "cleveland.csv"
COLUMNS = ["age","sex","cp","trestbps","chol","fbs","restecg",
           "thalach","exang","oldpeak","slope","ca","thal","target"]

raw = pd.read_csv(DATA_PATH, header=None)
raw.columns = COLUMNS

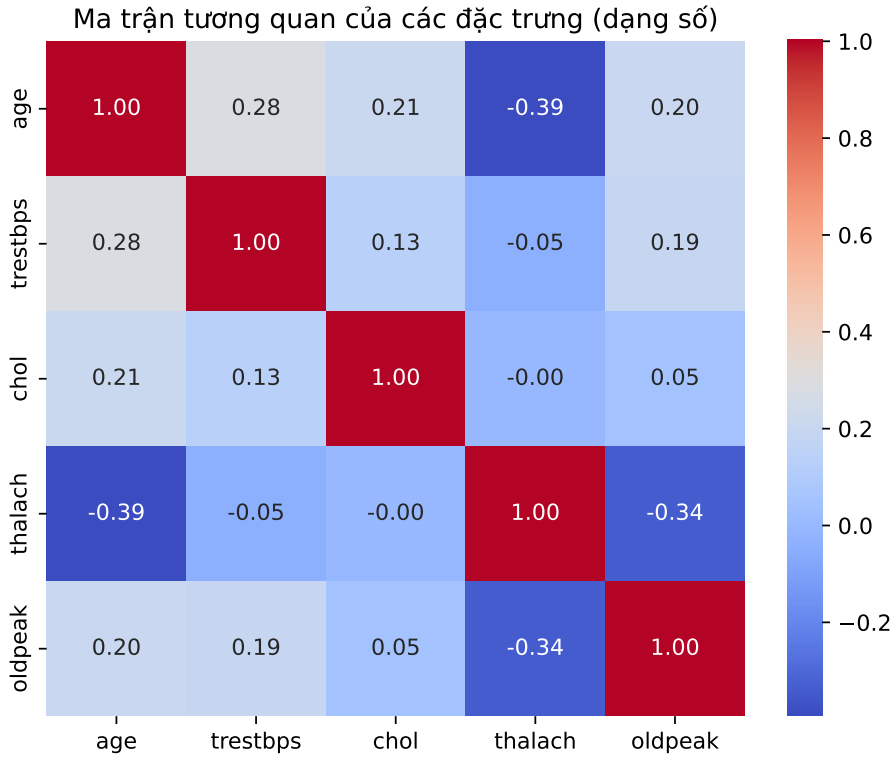
for c in ["age","trestbps","chol","thalach","oldpeak","ca","thal"]:
    raw[c] = pd.to_numeric(raw[c], errors="coerce")

raw["target"] = (raw["target"] > 0).astype(int)
print("Shape:", raw.shape)
display(raw.head())
display(raw.isna().sum())
```

Sau đó, tách danh sách cột dạng "số" và cột dạng "phân loại", làm cơ sở cho 2 luồng tiền xử lý khác nhau.

```
numeric_cols = ["age","trestbps","chol","thalach","oldpeak"]
categorical_cols = ["sex","cp","fbs","restecg","exang","slope","ca","thal"]
```

Tiếp đến là khám phá dữ liệu như trực quan ma trận tương quan, phân phối của các đặc trưng dạng số.

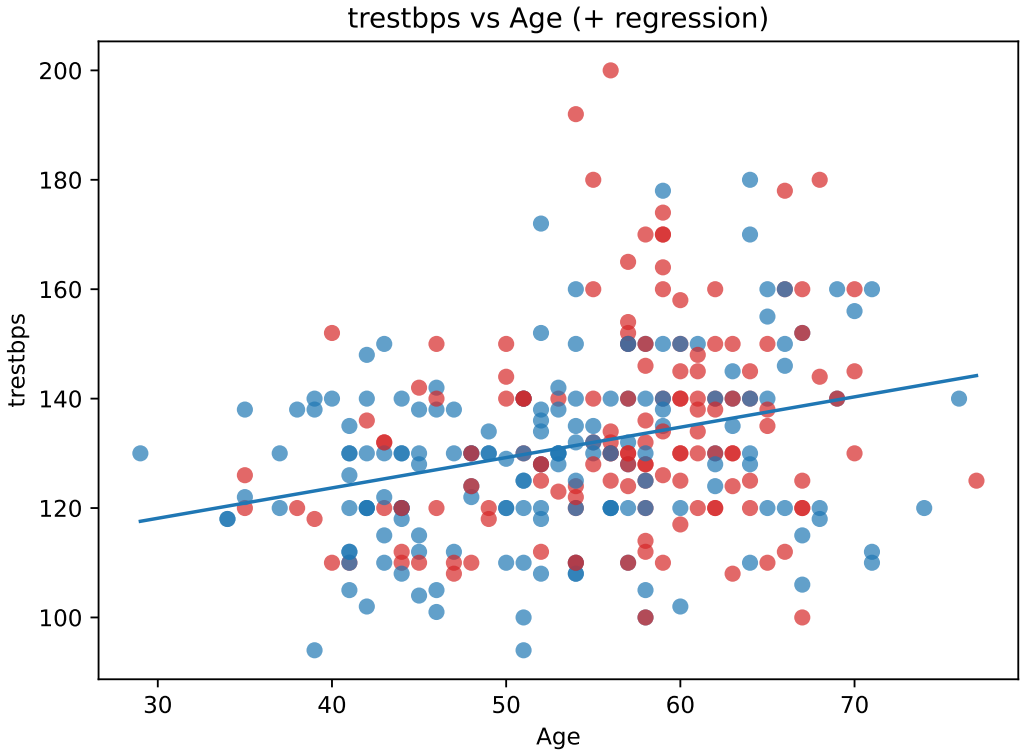


Hình 5: Ma trận tương quan giữa các đặc trưng.

```python
plt.figure(figsize=(6, 5))
corr_matrix = raw[numeric_cols].corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Ma trận tương quan của các đặc trưng (dạng số)")
plt.tight_layout()
plt.show()

sns.pairplot(raw, vars=numeric_cols, hue="target", diag_kind="kde")
plt.figure(figsize=(10, 10))
plt.suptitle("Phân phối theo từng cặp đặc trưng (dạng số) theo nhãn", y=1.02)
plt.savefig("pairwise_distribution.pdf", bbox_inches="tight")
plt.show()
```

Bên cạnh đó, tiến hành trực quan một số mối liên hệ như scatter age với các chỉ số (kèm đường hồi quy), xu hướng theo nhóm tuổi, và tương quan so với age, giúp gợi ý đặc trưng tiềm năng trước khi thực hiện tạo đặc trưng mới.



Hình 6: Mối quan hệ giữa tuổi và huyết áp tâm thu lúc nghỉ (trestbps), kèm đường hồi quy tuyến tính.

```
X, y = raw.drop(columns=["target"]), raw["target"]
colors = np.where(y==0, "tab:blue", "tab:red")
metrics = [c for c in ["chol","trestbps","thalach"] if c in X.columns]

def scatter_with_regression(xs, ys, xlab, ylab, title):
    m = xs.notna() & ys.notna()
    x, yy = xs[m].values, ys[m].values
    plt.figure()
    plt.scatter(xs, ys, c=colors, alpha=0.7)
    if len(x) > 1:
        k, b = np.polyfit(x, yy, 1)
        xline = np.linspace(np.nanmin(x), np.nanmax(x), 100)
        plt.plot(xline, k*xline + b)
    plt.xlabel(xlab)
    plt.ylabel(ylab)
    plt.title(title)
    plt.tight_layout()
    plt.show()

if "age" in X.columns and metrics:
    for col in metrics:
        scatter_with_regression(
            X["age"], X[col],
            "Age", col,
            f"{col} vs Age (+ regression)")
```

```
26
27 if "age" in X.columns and metrics:
28     age_bins = pd.cut(X["age"], bins=5)
29     for m in metrics:
30         plt.figure()
31         avg_vals = X.groupby(age_bins, observed=True)[m].mean()
32         plt.plot(range(len(avg_vals)), avg_vals, marker="o", label=f"Average {m}")
33         plt.xticks(
34             range(len(avg_vals)), map(str, avg_vals.index),
35             rotation=45, ha="right"
36         )
37         plt.xlabel("Age group")
38         plt.ylabel(m)
39         plt.title(f"{m} average per age group")
40         plt.legend()
41         plt.tight_layout()
42         plt.show()
43
44 cols = [c for c in ["age","chol","trestbps","thalach"] if c in X.columns]
45 if "age" in cols:
46     corr = X[cols].corr(numeric_only=True)["age"].drop("age", errors="ignore")
47     plt.figure(); corr.plot(kind="bar")
48     plt.ylabel("Correlation with age")
49     plt.title("Correlation of features with Age")
50     plt.tight_layout()
51     plt.show()
```

Chia bộ dữ liệu gốc thành 3 tập con train, val, test (tỉ lệ 80:10:10), sau đó biến đổi theo luồng xử lý như sau:

- Nếu cột là dạng "số": thực hiện thay thế các giá trị thiếu bằng giá trị trung vị (median) trong cột đó, sau đó chuẩn hóa dữ liệu số về phân phối có trung bình là 0 và độ lệch chuẩn bằng 1.
- Nếu cột là dạng "phân loại": thay thế giá trị thiếu bằng giá trị xuất hiện nhiều nhất (mode) trong cột đó, sau đó chuẩn hóa dữ liệu theo phương pháp Min-Max.

Sau đó lưu thành các tập gốc gồm `raw_train/val/test.csv`.

```
1 TARGET = "target"
2 feat_cols = raw.columns.drop(TARGET)
3 X_all, y_all = raw[feat_cols], raw[TARGET]
4 K = len(feat_cols)
5
6 num_proc = Pipeline([("imp", SimpleImputer(strategy="median")),
7                       ("sc", StandardScaler())])
8 cat_proc = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
9                       ("sc", MinMaxScaler())])
10
11 preprocess = ColumnTransformer([
12     ("num", num_proc, numeric_cols),
13     ("cat", cat_proc, categorical_cols),
```

```
], verbose_feature_names_out=False).set_output(transform="pandas")

raw_pipeline = Pipeline([("preprocess", preprocess)])

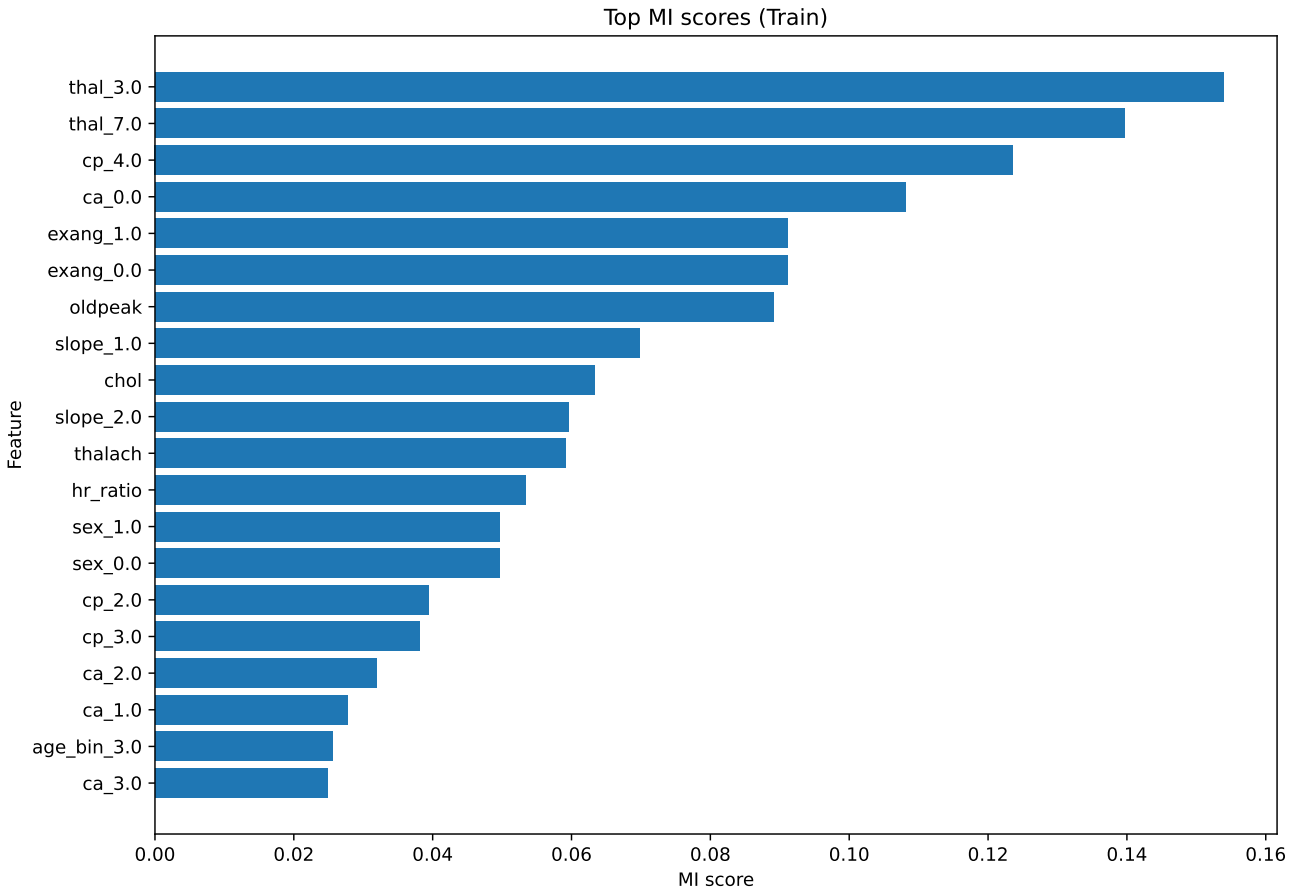
X_train, X_temp, y_train, y_temp = train_test_split(
    X_all, y_all, test_size=0.2, stratify=y_all, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42
)

X_raw_train = raw_pipeline.fit_transform(X_train, y_train)
X_raw_val = raw_pipeline.transform(X_val)
X_raw_test = raw_pipeline.transform(X_test)

out_dir = Path("splits"); out_dir.mkdir(parents=True, exist_ok=True)
for name, (X_, y_) in {
    "raw_train": (X_raw_train, y_train),
    "raw_val": (X_raw_val, y_val),
    "raw_test": (X_raw_test, y_test),
}.items():
    pd.concat(
        [X_, y_.rename(TARGET)], axis=1
    ).to_csv(out_dir / f"{name}.csv", index=False)

print(f"Saved RAW splits. K (RAW features) = {K}")
display(X_raw_train)
display(X_raw_train.isna().sum())
```

Cuối cùng, tạo bộ dữ liệu được xử lý đặc trưng bằng cách sinh thêm đặc trưng mới theo độ tuổi. Sau đó dữ, liệu được tiền xử lý theo hai luồng tương tự như trên tập gốc, nhưng các giá trị trong cột dạng "phân loại" được mã hóa theo One-hot. Sau đó, ước lượng mutual information (IM) để xếp hạng và chọn Top-K đặc trưng (với K bằng số lượng đặc trưng trên bộ gốc) và lưu thành các tập `fe_train/val/test.csv`.



Hình 7: Các đặc trưng quan trọng theo thang điểm Mutual Information (MI) trên tập huấn luyện.

```python
def add_new_features(df):
    df = df.copy()
    if {"chol","age"} <= set(df.columns):
        df["chol_per_age"] = df["chol"]/df["age"]
    if {"trestbps","age"} <= set(df.columns):
        df["bps_per_age"] = df["trestbps"]/df["age"]
    if {"thalach","age"} <= set(df.columns):
        df["hr_ratio"] = df["thalach"]/df["age"]
    if "age" in df.columns:
        df["age_bin"] = pd.cut(df["age"], bins=5, labels=False).astype("category")
    return df

gen_num = ["chol_per_age","bps_per_age","hr_ratio"]
gen_cat = ["age_bin"]
all_nums = [c for c in numeric_cols] + gen_num
all_cats = [c for c in categorical_cols] + gen_cat

num_proc = Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc",  StandardScaler())])
cat_proc = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False
```

```
                                        ))])

pre = ColumnTransformer([
    ("num", num_proc, all_nums),
    ("cat", cat_proc, all_cats),
], verbose_feature_names_out=False).set_output(transform="pandas")

fe_pre = Pipeline([
    ("add", FunctionTransformer(add_new_features, validate=False)),
    ("pre", pre),
]).set_output(transform="pandas")

Xt_tr = fe_pre.fit_transform(X_train, y_train)
Xt_va = fe_pre.transform(X_val)
Xt_te = fe_pre.transform(X_test)

nz_cols = Xt_tr.columns[Xt_tr.nunique(dropna=False) > 1]
Xt_tr, Xt_va, Xt_te = Xt_tr[nz_cols], Xt_va[nz_cols], Xt_te[nz_cols]

ohe = fe_pre.named_steps["pre"].named_transformers_["cat"].named_steps["ohe"]
cat_names = list(ohe.get_feature_names_out(all_cats))
is_discrete = np.array([c in cat_names for c in Xt_tr.columns], dtype=bool)

mi = mutual_info_classif(Xt_tr.values, y_train.values,
                         discrete_features=is_discrete, random_state=42)
mi_series = pd.Series(mi, index=Xt_tr.columns).sort_values(ascending=False)

K = raw.columns.drop("target").shape[0]
topk_cols = list(mi_series.head(K).index)

fe_tr = Xt_tr[topk_cols].assign(target=y_train.values)
fe_va = Xt_va[topk_cols].assign(target=y_val.values)
fe_te = Xt_te[topk_cols].assign(target=y_test.values)

out = Path("splits"); out.mkdir(parents=True, exist_ok=True)
fe_tr.to_csv(out/"fe_train.csv", index=False)
fe_va.to_csv(out/"fe_val.csv", index=False)
fe_te.to_csv(out/"fe_test.csv", index=False)

print(f"Saved FE splits. K (FE features) = {K}")
display(pd.Series(topk_cols, name="fe_topk_features").reset_index(drop=True))

N = min(20, len(mi_series))
topN = mi_series.head(N).iloc[::-1]
plt.figure(figsize=(10, max(6, 0.35*N)))
plt.barh(topN.index, topN.values)
plt.title("Top MI scores (Train)")
plt.xlabel("MI score")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Sau khi xử lý bộ dữ liệu với các bước trên, chạy code bên dưới để nén folder thành file ZIP

database.zip, trong đó gồm:

- Bộ dữ liệu gốc: 3 file `fe_train/val/test.csv`.
- Bộ dữ liệu bộ dữ liệu được xử lý đặc trưng: 3 file `fe_train/val/test.csv`.

```
!zip -r dataset.zip splits

# adding: splits/
# adding: splits/fe_train.csv
# adding: splits/raw_test.csv
# adding: splits/raw_val.csv
# adding: splits/fe_test.csv
# adding: splits/fe_val.csv
# adding: splits/raw_train.csv
```

II.3. Naive Bayes Classifier

II.3.1. Giới thiệu

Bắt đầu với Naive Bayes Classifier, một nhóm mô hình phân loại dựa trên định lý Bayes, tính toán xác suất hậu nghiệm của mỗi lớp và gán mẫu dữ liệu vào lớp có xác suất cao nhất.

$$\underbrace{P(A|B)}_{\text{Xác suất hậu nghiệm}} = \frac{\overbrace{P(B|A)}^{\text{Khả năng xảy ra}} \cdot \overbrace{P(A)}^{\text{Xác suất tiên nghiệm}}}{\underbrace{P(B)}_{\text{Bằng chứng}}}$$

Hình 8: Công thức Naive Bayes và các thành phần liên quan.

Để đơn giản hóa quá trình tính toán các xác suất phức tạp, mô hình áp dụng giả định "ngây thơ" (naive) rằng các đặc trưng đầu vào độc lập có điều kiện khi đã biết nhãn lớp. Nói cách khác, một khi biết bệnh nhân thuộc lớp "mắc bệnh tim" hay "không mắc bệnh tim", giá trị của một đặc trưng (ví dụ: cholesterol) sẽ không ảnh hưởng đến giá trị của đặc trưng khác (ví dụ: huyết áp) trong việc ước lượng xác suất thuộc lớp. Mặc dù giả định đơn giản, Naive Bayes vẫn cho thấy hiệu quả tốt trong nhiều trường hợp, đặc biệt khi dữ liệu có số lượng mẫu hạn chế và đặc trưng rời rạc.

II.3.2. Triển khai

Lưu ý: Các khối code được đánh dấu **"Tái sử dụng"** nghĩa là được sử dụng nhiều lần mà không cần sửa đổi giữa các mô hình khác nhau trong dự án này.

Bắt đầu bằng việc nạp các thư viện cần thiết và đặt seed để giữ kết quả nhất quán giữa các lần chạy.

Tái sử dụng

```python
# Import thư viện cần thiết
import os
import random
import warnings
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from collections import Counter
from sklearn.cluster import KMeans
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import StackingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score

warnings.filterwarnings("ignore")

# Đặt seed để đảm bảo kết quả nhất quán giữa các lần chạy
SEED = 42
os.environ["PYTHONHASHSEED"] = str(SEED)
np.random.seed(SEED)
random.seed(SEED)
print(f"Seed: {SEED}")
```

Tiếp theo, tải bộ dữ liệu đã được xử lý như ở mục II.2., khi này bộ dữ liệu gồm 2 loại: bộ gốc và bộ được xử lý đặc trưng (được đánh dấu bằng ký hiệu "fe"), mỗi bộ gồm các tập train, val, test đã được chia sẵn.

Tái sử dụng

```
!gdown 1T6AWCoyeC2MqGvmqZPPcA4ni5Md1k0sI
!unzip dataset.zip
```

Kế tiếp, xây dựng một hàm đọc file CSV, tách các đặc trưng X và nhãn y để phục vụ cho việc huấn luyện mô hình, ngoài ra, hiển thị thêm một số thông tin cơ bản của tập dữ liệu.

Tái sử dụng

```python
def read_csv(file_path):
    df = pd.read_csv(file_path)
```

```
    display(df.head())
    X = df.drop("target", axis=1)
    y = df["target"]

    display(y.value_counts())
    print("Shape df: ", df.shape)
    print("Shape X: ", X.shape)
    print("Shape y: ", y.shape)
    return X, y
```

Sau đó, lần lượt đọc các tập dữ liệu gốc: `raw_train/val/test`

Tái sử dụng

```
# Tải các bộ dữ liệu gốc đã được xử lý
X_train, y_train = read_csv("splits/raw_train.csv")
X_val, y_val = read_csv("splits/raw_val.csv")
X_test, y_test = read_csv("splits/raw_test.csv")
```

và các tập fe: `fe_train/val/test` để chuẩn bị cho những thực nghiệm bên dưới.

Tái sử dụng

```
# Tải các bộ dữ liệu được áp dụng kỹ thuật tạo đặc trưng
X_train_fe, y_train_fe = read_csv("splits/fe_train.csv")
X_val_fe, y_val_fe = read_csv("splits/fe_val.csv")
X_test_fe, y_test_fe = read_csv("splits/fe_test.csv")
```

Theo sau là 2 hàm `evaluate_val` và `evaluate_test`, mục đích chung của 2 này trong toàn bộ các mô hình trong dự án này là:

- Hàm `evaluate_val`: huấn luyện trên tập train và đánh giá trên tập val,
- Hàm `evaluate_test`: đánh giá mô hình đã huấn luyện trên tập test.

```
def evaluate_val(X_train, y_train, X_val, y_val):
    # Huấn luyện Naive Bayes
    nbc_model = GaussianNB()
    nbc_model.fit(X_train, y_train)

    # Dự đoán và đánh giá trên tập val
    nb_pred = nbc_model.predict(X_val)
    nb_accuracy = accuracy_score(y_val, nb_pred)

    print(f"Độ chính xác Naive Bayes: {nb_accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_val, nb_pred))
    return nbc_model, nb_accuracy

```

```python
def evaluate_test(nbc_model, X_test, y_test):
    # Dự đoán và đánh giá trên tập test
    nb_test_pred = nbc_model.predict(X_test)
    nb_test_accuracy = accuracy_score(y_test, nb_test_pred)

    print(f"Độ chính xác Naive Bayes trên tập test: {nb_test_accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, nb_test_pred))
    return nb_test_accuracy
```

Khi này, tiến hành huấn luyện và đánh giá Naive Bayes trên các tập gốc, lưu lại độ chính xác trên val và test.

```python
# Huấn luyện và đánh giá với dữ liệu gốc
model, accuracy = evaluate_val(X_train, y_train, X_val, y_val)
test_accuracy = evaluate_test(model, X_test, y_test)
```

Tương tự như bước trên cho tập fe.

```python
# Huấn luyện và đánh giá với kỹ thuật tạo đặc trưng
model_fe, accuracy_fe = evaluate_val(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)
```

Cuối cùng, vẽ biểu đồ cột so sánh độ chính xác (acc) trên tập val và test cho hai kịch bản (tập gốc và tập fe).

Tái sử dụng

```python
plt.rcParams["font.family"] = "DejaVu Serif"
labels = ["Original Dataset", "Feature Engineering Dataset"]
val_accs  = [accuracy, accuracy_fe]
test_accs = [test_accuracy, test_accuracy_fe]

x = np.arange(len(labels))
width = 0.3

fig, ax = plt.subplots(figsize=(5, 5))
rects1 = ax.bar(x - width/2, val_accs,  width,
                label="Validation Accuracy",
                color="tab:blue", edgecolor="black", linewidth=1.2)
rects2 = ax.bar(x + width/2, test_accs, width,
                label="Test Accuracy",
                color="tab:red", edgecolor="black", linewidth=1.2)

ax.set_ylim(0.5, 1.05)
ax.set_ylabel("Accuracy")
ax.set_title("Acc trên tập Val và Test")
ax.set_xticks(x)
```

```
ax.set_xticklabels(labels)
ax.legend(ncol=2, loc="upper center")

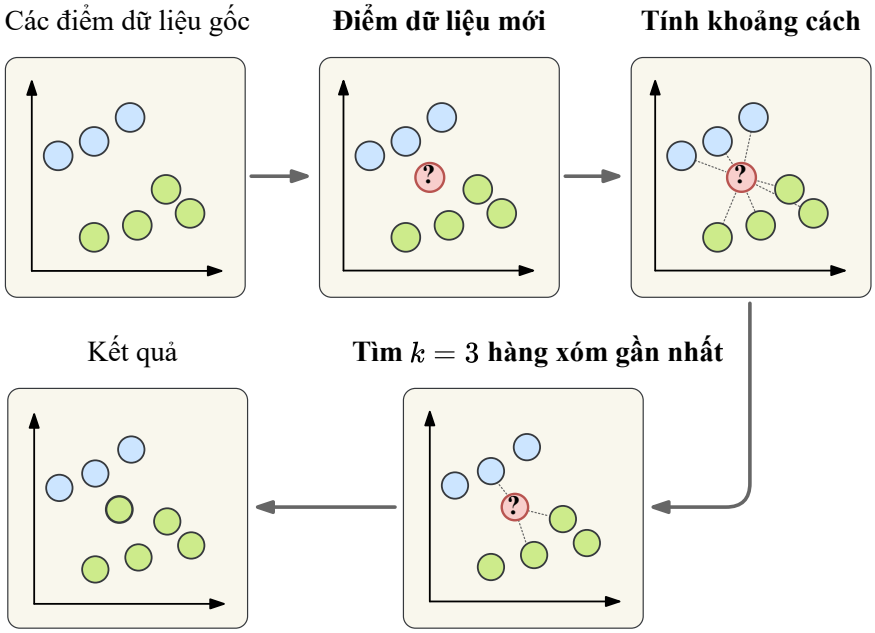
def autolabel(rects):
    for rect in rects:
        h = rect.get_height()
        ax.annotate(f"{h:.2f}", xy=(rect.get_x()+rect.get_width()/2, h),
                    ha="center", va="bottom")

autolabel(rects1)
autolabel(rects2)

fig.tight_layout()
plt.show()
```

II.4. K-Nearest Neighbors

II.4.1. Giới thiệu



Hình 9: Tổng quan các bước trong mô hình KNN.

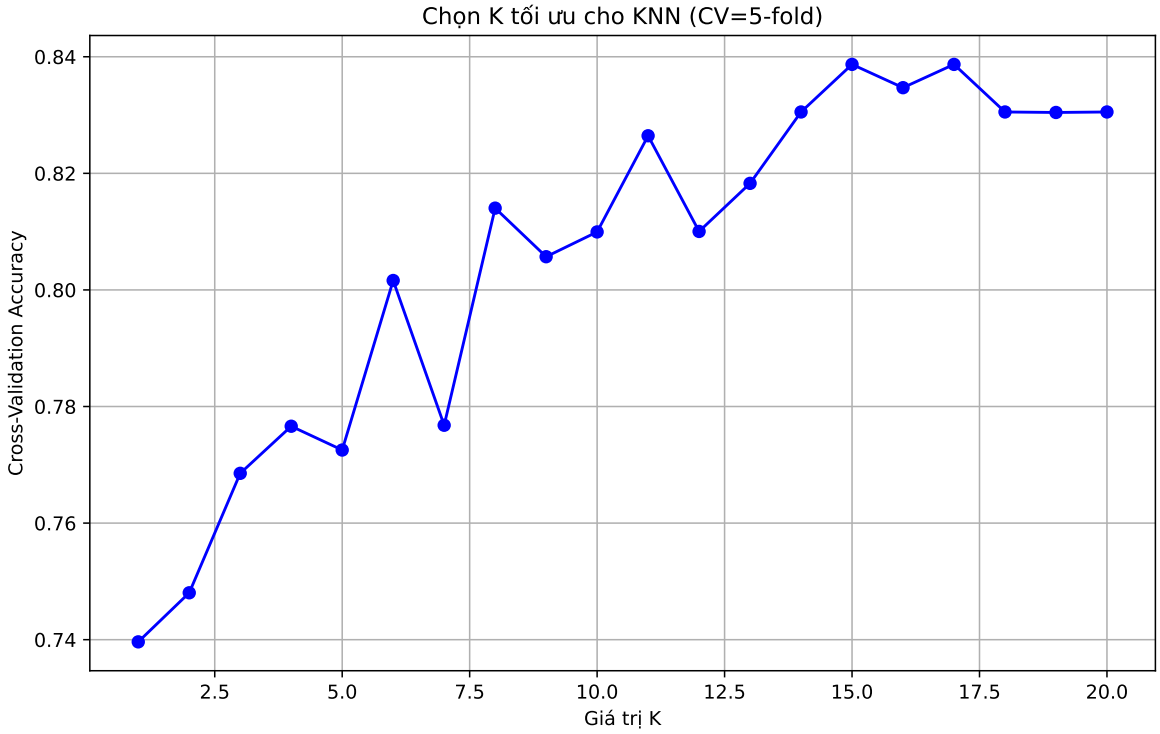
Tiếp theo là K-Nearest Neighbors (KNN), một mô hình học có giám sát và phi tham số (non-parametric). Thuật toán này không học một hàm mục tiêu cụ thể, mà lưu trữ toàn bộ tập huấn luyện để sử dụng khi dự đoán. Nguyên tắc hoạt động dựa trên giả định rằng các điểm dữ liệu có đặc trưng tương tự nhau sẽ có xu hướng nằm gần nhau trong không gian đặc trưng. Khi phân loại một mẫu mới, KNN xác định K điểm gần nhất trong tập huấn luyện (các "hàng xóm") dựa trên một thước đo khoảng cách, sau đó gán nhãn dựa trên đa số phiếu của các hàng xóm này.

Lựa chọn giá trị K là yếu tố quan trọng: K quá nhỏ khiến mô hình nhạy cảm với nhiễu, trong khi K quá lớn có thể làm mờ ranh giới giữa các lớp.

II.4.2. Triển khai

Với mô hình KNN, tái sử dụng các khối code cũ và điều chỉnh một số hàm chính như sau:

- Bổ sung hàm tìm k tối ưu theo phương pháp Stratified K-Fold Cross Validation,
- Điều chỉnh hàm `evaluate_val` và `evaluate_test` cho KNN.



Hình 10: Biểu đồ xác định K tối ưu cho KNN (CV=5-fold).

```python
def find_optimal_k(X_train, y_train, X_val=None, y_val=None,
                   k_range=range(1, 21), cv_splits=5):
    # Tạo bộ chia dữ liệu cross-validation
    cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
    k_scores = []

    # Đánh giá bằng cross-validation trên tập train
    for k in k_range:
        knn = KNeighborsClassifier(n_neighbors=k)
        cv_score = cross_val_score(
            knn, X_train, y_train,
            cv=cv, scoring="accuracy", n_jobs=-1)
        k_scores.append(cv_score.mean())

    # Vẽ biểu đồ chọn k theo CV
```

```
    plt.figure(figsize=(10, 6))
    plt.plot(list(k_range), k_scores, "bo-")
    plt.title(f"Chọn K tối ưu cho KNN (CV={cv_splits}-fold)")
    plt.xlabel("Giá trị K")
    plt.ylabel("Cross-Validation Accuracy")
    plt.grid(True)
    plt.show()

    # K tối ưu theo CV
    optimal_k = list(k_range)[int(np.argmax(k_scores))]
    print(f"K tối ưu (CV): {optimal_k}")
    return optimal_k

def evaluate_val(X_train, y_train, X_val, y_val, optimal_k):
    # Huấn luyện KNN với k tối ưu
    print(f"Huấn luyện KNN với k tối ưu: {optimal_k}")
    knn_model = KNeighborsClassifier(n_neighbors=optimal_k)
    knn_model.fit(X_train, y_train)

    # Dự đoán và đánh giá trên tập val
    knn_pred = knn_model.predict(X_val)
    knn_accuracy = accuracy_score(y_val, knn_pred)

    print(f"\nĐộ chính xác KNN trên tập validation: {knn_accuracy:.2f}")
    print("Classification Report:")
    print(classification_report(y_val, knn_pred))
    return knn_model, knn_accuracy

def evaluate_test(knn_model, X_test, y_test):
    # Dự đoán và đánh giá trên tập test
    knn_test_pred = knn_model.predict(X_test)
    knn_test_accuracy = accuracy_score(y_test, knn_test_pred)

    print(f"\nĐộ chính xác KNN trên tập test: {knn_test_accuracy:.2f}")
    print("Classification Report:")
    print(classification_report(y_test, knn_test_pred))
    return knn_test_accuracy
```

Trong phần huấn luyện mô hình, thực hiện tương tự như mô hình trước nhưng bổ sung thêm việc tìm k tối ưu và sau đó huấn luyện mô hình KNN với giá trị k này.

```
k_optimal = find_optimal_k(X_train, y_train, X_val, y_val)
model, accuracy = evaluate_val(X_train, y_train, X_val, y_val, k_optimal)
test_accuracy = evaluate_test(model, X_test, y_test)

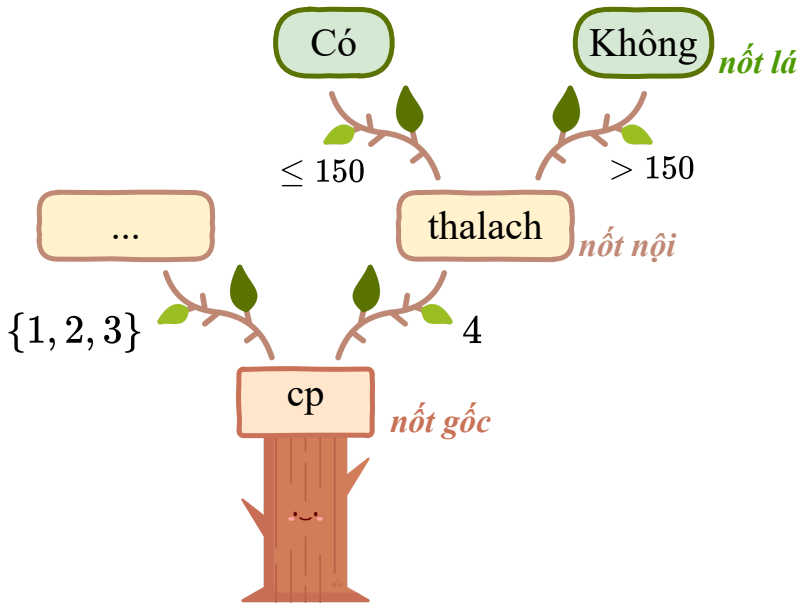
###

k_optimal = find_optimal_k(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
model_fe, accuracy_fe = evaluate_val(
    X_train_fe, y_train_fe,
    X_val_fe, y_val_fe,
```

```
    k_optimal
)
test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)
```

II.5. Decision Tree

II.5.1. Giới thiệu



Hình 11: Minh họa mô hình cây quyết định bằng các thuộc tính của bài toán dự đoán bệnh tim.

Sau KNN, chúng ta chuyển sang Decision Tree, một mô hình học có giám sát sử dụng cấu trúc dạng cây để đưa ra quyết định phân loại. Mô hình chia nhỏ không gian đặc trưng thành các vùng bằng cách áp dụng các quy tắc "nếu-thì" tại mỗi nút. Mỗi **nút nội** biểu diễn một phép kiểm tra trên một đặc trưng, mỗi nhánh thể hiện kết quả của phép kiểm tra đó, và mỗi **nút lá** đại diện cho nhãn dự đoán. Việc xây dựng cây dựa trên việc chọn đặc trưng và ngưỡng phân chia tối ưu tại mỗi bước, nhằm tối đa hóa độ thuần khiết của các nút con, thường đo bằng các chỉ số như **Gini Impurity** hoặc **Information Gain**. Nhờ tính trực quan và khả năng diễn giải cao, Decision Tree là một trong những mô hình nền tảng và dễ hiểu nhất trong học máy.

II.5.2. Triển khai

Với mô hình DT, tái sử dụng các khối code cũ và điều chỉnh một số hàm chính như sau:

- Bổ sung hàm tìm độ sâu (Depth) tối ưu theo phương pháp Stratified K-Fold Cross Validation.
- Điều chỉnh hàm `evaluate_val` và `evaluate_test` cho DT.
- Bổ sung hàm hiển thị các đặc trưng có ảnh hưởng với mô hình.

```python
def find_optimal_depth(X_train, y_train,
                       depth_range=range(1, 11), cv_splits=5, min_depth=2):
    # Tạo bộ chia dữ liệu cross-validation
    cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
    depth_scores = []

    # Đánh giá bằng cross-validation trên tập train
    for depth in range(min_depth, depth_range.stop):
        dt = DecisionTreeClassifier(max_depth=depth, random_state=SEED)
        cv_score = cross_val_score(
            dt, X_train, y_train,
            cv=cv, scoring="accuracy", n_jobs=-1)
        depth_scores.append(cv_score.mean())

    # Vẽ biểu đồ chọn k theo CV
    plt.figure(figsize=(10, 6))
    plt.plot(list(range(min_depth, depth_range.stop)), depth_scores, "bo-")
    plt.title(f"Chọn Depth tối ưu cho DT (CV={cv_splits}-fold)")
    plt.xlabel("Giá trị Depth")
    plt.ylabel("Cross-Validation Accuracy")
    plt.grid(True)
    plt.show()

    # K tối ưu theo CV
    optimal_depth = list(range(min_depth, depth_range.stop))[int(np.argmax(depth_scores
                                                                           ))]
    print(f"Depth tối ưu (CV): {optimal_depth}")
    return optimal_depth

def evaluate_val(X_train, y_train, X_val, y_val, optimal_depth):
    # Huấn luyện với độ sâu tối ưu
    dt_model = DecisionTreeClassifier(max_depth=optimal_depth, random_state=SEED)
    dt_model.fit(X_train, y_train)

    # Dự đoán và đánh giá trên tập val
    dt_pred = dt_model.predict(X_val)
    dt_accuracy = accuracy_score(y_val, dt_pred)
    print(f"\nĐộ chính xác Decision Tree trên tập validation: {dt_accuracy:.2f}")

    print("Classification Report:")
    print(classification_report(y_val, dt_pred))
    return dt_model, dt_accuracy

def evaluate_test(dt_model, X_test, y_test):
    # Dự đoán và đánh giá trên tập test
    dt_test_pred = dt_model.predict(X_test)
    dt_test_accuracy = accuracy_score(y_test, dt_test_pred)
    print(f"\nĐộ chính xác Decision Tree trên tập test: {dt_test_accuracy:.2f}")

    print("Classification Report:")
    print(classification_report(y_test, dt_test_pred))
    return dt_test_accuracy

def display_feature_importance(dt_model, X_train):
```

```
    # Hiển thị các đặc trưng có ảnh hưởng lớn
    feature_importance = pd.DataFrame({
        "feature": X_train.columns,
        "importance": dt_model.feature_importances_
    }).sort_values("importance", ascending=False)
    print("\nXếp hạng Features quan trọng:")
    display(feature_importance.head(10))
```

Phần huấn luyện này tương tự KNN thực hiện tương tự như mô hình trước nhưng bổ sung thêm việc tìm k tối ưu và sau đó huấn luyện mô hình KNN với giá trị k này.

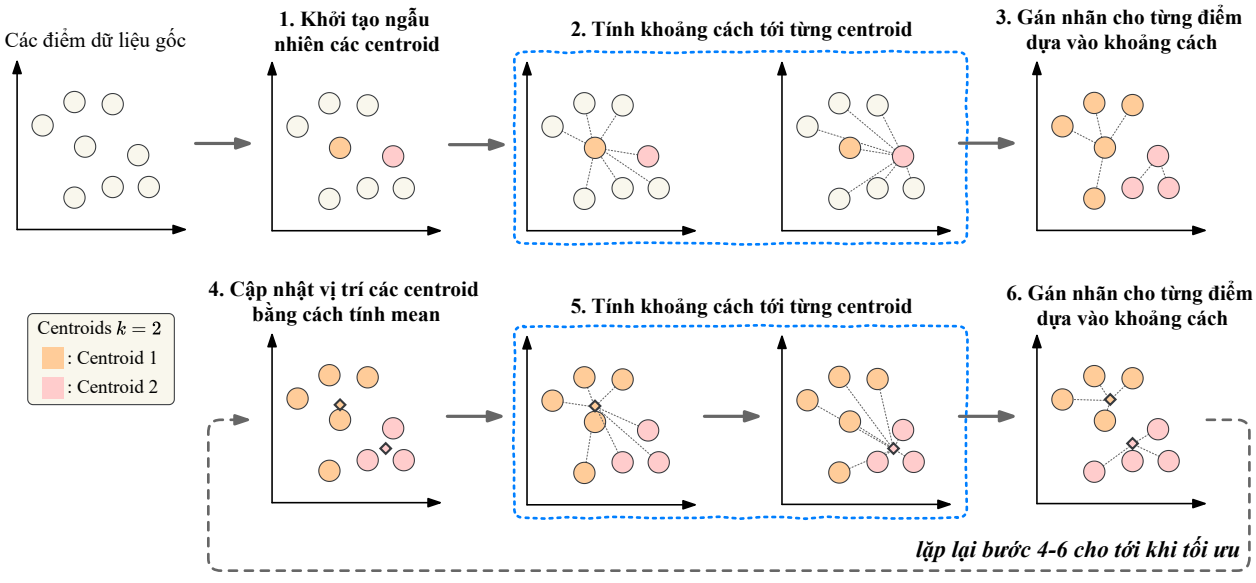
```
depth_optimal = find_optimal_depth(X_train, y_train, X_val, y_val)
model, accuracy = evaluate_val(X_train, y_train, X_val, y_val, depth_optimal)
test_accuracy = evaluate_test(model, X_test, y_test)
display_feature_importance(model, X_train)

###

depth_optimal = find_optimal_depth(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
model_fe, accuracy_fe = evaluate_val(X_train_fe, y_train_fe, X_val_fe, y_val_fe,
                                     depth_optimal)
test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)
display_feature_importance(model_fe, X_train_fe)
```

II.6. K-means

II.6.1. Giới thiệu



Hình 12: Minh họa từng bước hoạt động của thuật toán k-means với $k = 2$.

Khác với các mô hình học có giám sát bên trên, K-means là một thuật toán học không giám sát được sử dụng phổ biến cho bài toán phân cụm dữ liệu. Ý tưởng chính của K-means là nhóm các điểm dữ liệu thành K cụm sao cho các điểm trong cùng một cụm có đặc trưng tương tự nhau và khác biệt so với các cụm khác.

Thuật toán bắt đầu bằng việc khởi tạo ngẫu nhiên K điểm làm centroid (tâm cụm), sau đó lặp lại hai bước chính:

1. Gán mỗi điểm dữ liệu vào cụm có centroid gần nhất và
2. cập nhật lại vị trí của các centroid dựa trên trung bình các điểm trong cụm.

Quá trình này tiếp tục cho đến khi vị trí các centroid hội tụ hoặc đạt điều kiện dừng.

II.6.2. Triển khai

```python
def evaluate_val(X_train, y_train, X_val, y_val):
    # Huấn luyện
    kmean_model = KMeans(n_clusters=2, random_state=SEED, init="random")
    kmean_model.fit(X_train)

    # Dự đoán và đánh giá trên tập val
    val_clusters = kmean_model.predict(X_val)
    train_clusters = kmean_model.labels_
    cluster_class_mapping = {
        cluster_id: Counter(y_train[train_clusters == cluster_id]).most_common(1)[0][0]
        for cluster_id in np.unique(train_clusters)
    }
    kmeans_pred = np.array([cluster_class_mapping[cluster_id]
                            for cluster_id in val_clusters])
    kmeans_accuracy = accuracy_score(y_val, kmeans_pred)

    print(f"Độ chính xác KMeans (Binary Prediction): {kmeans_accuracy:.4f}")
    print("\nClassification Report (KMeans Binary Prediction):")
    print(classification_report(y_val, kmeans_pred))
    return kmean_model, kmeans_accuracy

def evaluate_test(kmean_model, X_test, y_test):
    # Dự đoán trên tập test
    test_clusters = kmean_model.predict(X_test)
    train_clusters = kmean_model.labels_
    cluster_class_mapping = {
        cluster_id: Counter(y_train[train_clusters == cluster_id]).most_common(1)[0][0]
        for cluster_id in np.unique(train_clusters)
    }
    kmeans_test_pred = np.array([cluster_class_mapping[cluster_id]
                                 for cluster_id in test_clusters])
    kmeans_test_accuracy = accuracy_score(y_test, kmeans_test_pred)
    print(f"Độ chính xác KMeans trên tập test: {kmeans_test_accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, kmeans_test_pred))
    return kmeans_test_accuracy
```

Phần huấn luyện K-Means thực hiện tương tự như các phần trước.

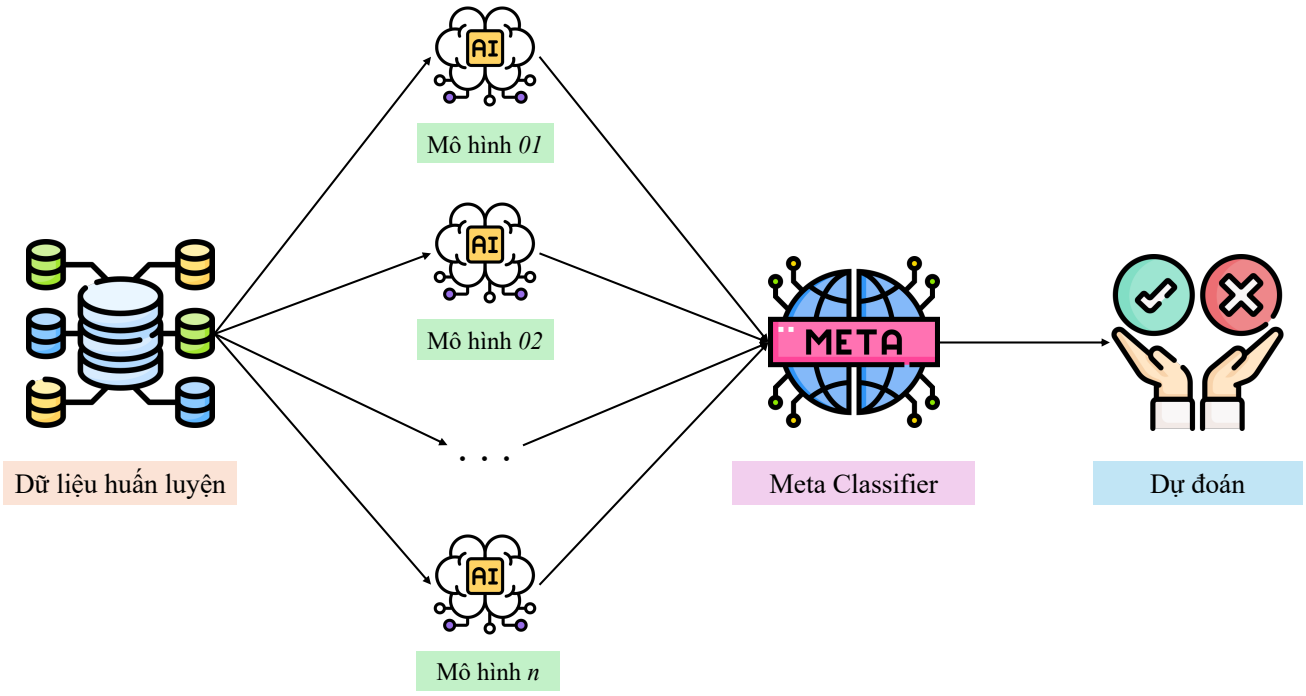
```
model, accuracy = evaluate_val(X_train, y_train, X_val, y_val)
test_accuracy = evaluate_test(model, X_test, y_test)

model_fe, accuracy_fe = evaluate_val(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)
```

II.7. Cải tiến: Ensemble

II.7.1. Giới thiệu

Ensemble learning là một kỹ thuật trong học máy nhằm kết hợp nhiều mô hình dự đoán để tạo thành một mô hình mạnh hơn. Ý tưởng chính là tận dụng ưu điểm của từng mô hình đơn lẻ, từ đó giảm sai số và tăng độ ổn định trong quá trình dự đoán. Chính bởi lẽ đó, chúng ta có thể tận dụng các mô hình đã huấn luyện từ trước trong project để ứng dụng kỹ thuật này nhằm làm tăng độ chính xác tổng thể của toàn hệ thống.



Hình 13: Minh họa ý tưởng cơ bản của kỹ thuật ensemble learning tận dụng n mô hình AI đã được huấn luyện.

Ensemble learning bao gồm rất nhiều những kỹ thuật con xoay quanh, song trong phạm vi của project này, chúng ta sử dụng phương pháp Stacking, kết hợp ba mô hình cơ bản (KNN, Decision Tree và Naive Bayes) và dùng một bộ phân loại cuối cùng để đưa ra kết quả. Cách tiếp cận này được kỳ vọng sẽ cải thiện hiệu năng so với từng mô hình riêng lẻ.

II.7.2. Triển khai

Đầu tiên, chúng ta khởi tạo mô hình Stacking Ensemble, trong đó ba mô hình con (KNN, Decision Tree và Naive Bayes) đóng vai trò là các bộ phân loại cơ sở. Mô hình KNN được chọn làm bộ phân loại cuối cùng để tổng hợp kết quả.

```
def init_model():
    # Khởi tạo mô hình Stacking Ensemble
    ensemble_model = StackingClassifier(
        estimators=[
            ("knn", KNeighborsClassifier()),
            ("dt", DecisionTreeClassifier(random_state=SEED)),
            ("nb", GaussianNB()),
        ],
        final_estimator=KNeighborsClassifier(),
        stack_method="predict_proba",
        passthrough=False
    )
    display(ensemble_model)
    return ensemble_model
```

Tiếp theo, chúng ta định nghĩa các hàm đánh giá, bao gồm huấn luyện và kiểm tra trên tập validation, cũng như đánh giá hiệu năng cuối cùng trên tập test. Các hàm này trả về độ chính xác cùng báo cáo chi tiết về precision, recall và F1-score.

```
def evaluate_ensemble(ensemble_model, X_train, y_train, X_val, y_val):
    ensemble_model.fit(X_train, y_train)
    ensemble_pred = ensemble_model.predict(X_val)
    ensemble_accuracy = accuracy_score(y_val, ensemble_pred)
    print(f"Độ chính xác Ensemble Model: {ensemble_accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_val, ensemble_pred))
    return ensemble_accuracy

def evaluate_ensemble_test(ensemble_model, X_test, y_test):
    ensemble_test_pred = ensemble_model.predict(X_test)
    ensemble_test_accuracy = accuracy_score(y_test, ensemble_test_pred)
    print(f"Độ chính xác Ensemble trên tập test: {ensemble_test_accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, ensemble_test_pred))
    return ensemble_test_accuracy
```

Sau khi đã xây dựng các hàm, chúng ta tiến hành huấn luyện mô hình Ensemble trên bộ dữ liệu gốc, đồng thời đánh giá hiệu năng trên cả tập validation và tập test.

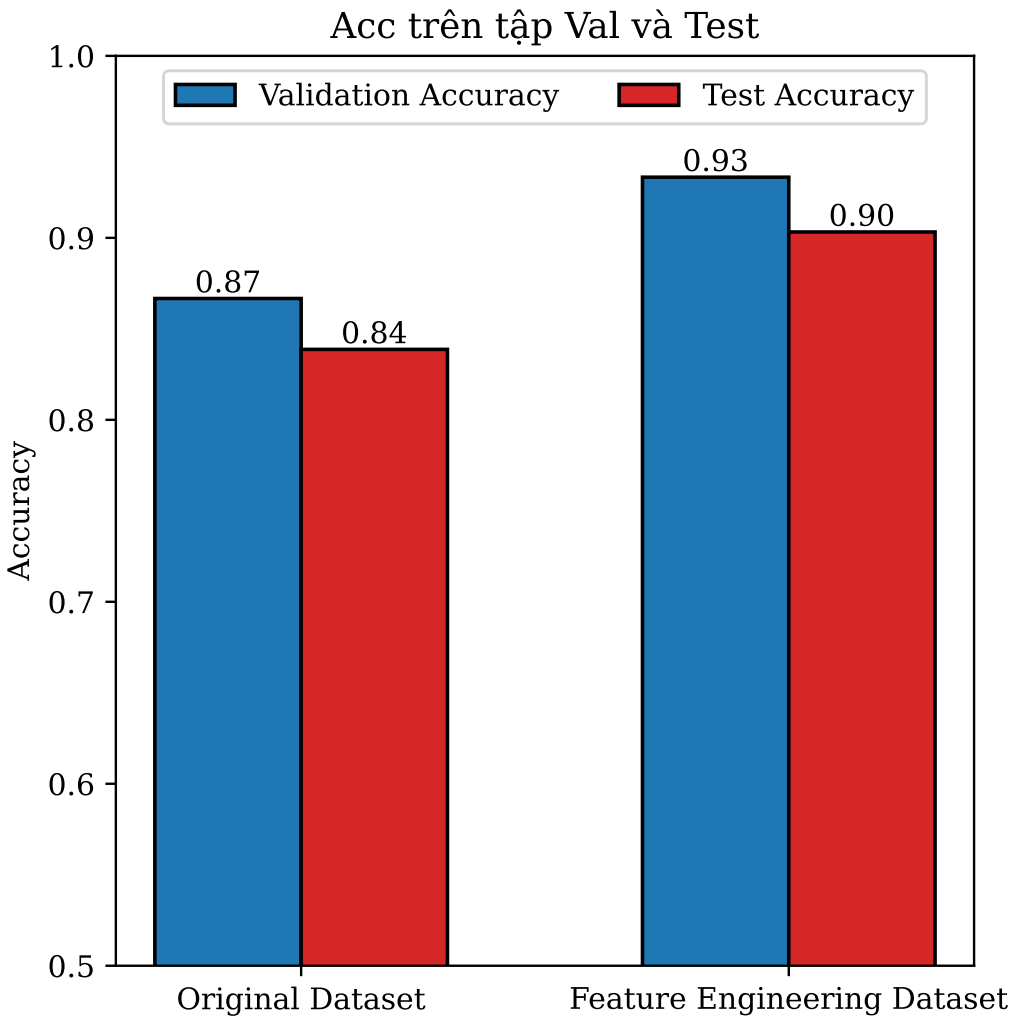
```
ensemble_model = init_model()
accuracy = evaluate_ensemble(
    ensemble_model,
    X_train,
```

```
    y_train,
    X_val,
    y_val
)
test_accuracy = evaluate_ensemble_test(
    ensemble_model,
    X_test,
    y_test
)
```

Tương tự, chúng ta cũng lặp lại quy trình với bộ dữ liệu đã được xử lý đặc trưng (feature engineering). Điều này giúp đánh giá mức độ cải thiện hiệu năng của kỹ thuật ensemble khi dữ liệu đầu vào được tinh chỉnh tốt hơn.

```
ensemble_model_fe = init_model()
accuracy_fe = evaluate_ensemble(
    ensemble_model_fe,
    X_train_fe,
    y_train_fe,
    X_val_fe,
    y_val_fe
)
test_accuracy_fe = evaluate_ensemble_test(
    ensemble_model_fe,
    X_test_fe,
    y_test_fe
)
```

Biểu đồ so sánh ở Hình 14 minh họa sự khác biệt về độ chính xác giữa hai cách tiếp cận: sử dụng dữ liệu gốc và dữ liệu đã được xử lý đặc trưng. Có thể nhận thấy rằng việc áp dụng kỹ thuật feature engineering kết hợp với Ensemble giúp cải thiện hiệu năng một cách rõ rệt.



Hình 14: Biểu đồ so sánh mô hình Ensemble (Stacking) trên bộ dữ liệu gốc và bộ được xử lý đặc trưng.

III. Câu hỏi trắc nghiệm

1. Trong phần Naive Bayes Classifier, mô hình giả định rằng:
   (a) Tất cả đặc trưng y tế đều phụ thuộc tuyến tính vào tuổi.
   (b) Mọi đặc trưng phải được chuẩn hóa trước khi tính toán.
   (c) Cholesterol và huyết áp độc lập có điều kiện khi biết bệnh nhân mắc hay không mắc bệnh tim.
   (d) Mỗi đặc trưng chỉ có thể nhận 2 giá trị rời rạc.

2. Trong phần cài đặt code KNN, giá trị K được chọn bằng cách:
   (a) Dùng GridSearchCV trên toàn bộ dữ liệu.
   (b) Chọn K bằng cách thử nghiệm ngẫu nhiên nhiều lần.
   (c) Luôn cố định K = 5 theo tài liệu tham khảo.
   (d) Tối ưu bằng Stratified K-Fold Cross Validation với k từ 1 đến 20.

3. Khi huấn luyện Decision Tree trong bài, tác giả dùng chỉ số nào để chọn thuộc tính chia tách?
   (a) Entropy hoặc Gini Impurity để tối đa hóa Information Gain.
   (b) Mean Squared Error vì đây là bài toán hồi quy.
   (c) Cosine Similarity để so sánh đặc trưng.
   (d) Precision-Recall Curve trên tập validation.

4. Với việc thuật toán K-means được cài đặt số cụm bằng 2, điều này phản ánh ý nghĩa gì?
   (a) Hai trạng thái sức khỏe tim mạch: có bệnh và không bệnh.
   (b) Hai nhóm bệnh nhân nam và nữ.
   (c) Hai loại chỉ số lâm sàng: cholesterol và huyết áp.
   (d) Hai bộ dữ liệu raw và feature engineering.

5. Trong phần Ensemble, bài viết sử dụng `StackingClassifier` với ba mô hình con là:
   (a) Logistic Regression, SVM, Random Forest.
   (b) KNN, Naive Bayes, K-means.
   (c) Decision Tree, K-means, SVM.
   (d) KNN, Decision Tree, Naive Bayes.

6. Trong phần Decision Tree, hàm `display_feature_importance()` có vai trò gì?
   (a) Hiển thị phân phối nhãn trong tập huấn luyện.
   (b) Xếp hạng các đặc trưng quan trọng nhất dựa trên độ đóng góp vào việc phân loại.
   (c) Tự động chọn độ sâu tối ưu cho cây.

(d) Chuẩn hóa các đặc trưng đầu vào trước khi huấn luyện.

7. Trong tập dữ liệu Cleveland, thuộc tính `ca` có ý nghĩa gì?

   (a) Loại đau ngực (1–4).

   (b) Thalassemia (3,6,7).

   (c) Điện tâm đồ lúc nghỉ (0–2).

   (d) Số mạch máu chính được nhuộm màu (0–3).

8. Đoạn code sau trong phần KNN có mục đích gì?

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

   (a) Khởi tạo mô hình KNN với k=5.

   (b) Chia dữ liệu thành 5 folds cân bằng theo nhãn để cross-validation, giữ kết quả nhất quán nhờ seed.

   (c) Ngăn chặn overfitting bằng cách bỏ bớt dữ liệu.

   (d) Chuẩn hóa các đặc trưng về cùng khoảng giá trị.

9. Trong phần Naive Bayes, lệnh `classification_report(y_val, nb_pred)` trả về:

   (a) Precision, Recall, F1-score cho từng lớp (mắc bệnh tim / không mắc bệnh tim).

   (b) Chỉ duy nhất Accuracy trên toàn bộ tập validation.

   (c) Biểu đồ histogram của dự đoán.

   (d) Giá trị K tối ưu được chọn.

10. Trong triển khai K-means, đoạn code:

```
cluster_class_mapping = {
    cluster_id: Counter(y_train[train_clusters == cluster_id]).most_common(1)[0][0]
    for cluster_id in np.unique(train_clusters)
}
```

có ý nghĩa là:

   (a) Ánh xạ mỗi cụm thành nhãn "có bệnh tim" hoặc "không bệnh" dựa trên nhãn chiếm đa số trong cụm train.

   (b) Chuẩn hóa dữ liệu trước khi phân cụm.

   (c) Tự động chọn số cụm tối ưu.

   (d) Loại bỏ các điểm ngoại lai trong tập train.

IV. Tài liệu tham khảo

[1] PCR, *World Heart Day 2023: Reducing the burden of cardiovascular disease globally: Beyond stents and balloons!*, en. [Online]. Available: https://www.pcronline.com/News/Whats-new-on-PCRonline/2023/World-Heart-Day-2023-Reducing-burden-cardiovascular-disease-globally-beyond-stents-balloons.

[2] *Cardiovascular Disease | NCD Alliance*, en, Oct. 2023. [Online]. Available: https://ncdalliance.org/explore-ncds/ncds/cardiovascular-disease.

[3] *World Heart Report 2023: Full Report*, en-GB. [Online]. Available: https://world-heart-federation.org/resource/world-heart-report-2023/.

[4] A. O. Mocumbi, "Cardiovascular Health Care in Low- and Middle-Income Countries", *Circulation*, vol. 149, no. 8, pp. 557–559, Feb. 2024, Publisher: American Heart Association. DOI: 10.1161/CIRCULATIONAHA.123.065717. [Online]. Available: https://www.ahajournals.org/doi/10.1161/CIRCULATIONAHA.123.065717.

[5] P. Shinde et al., "A Survey on Machine Learning Techniques for Heart Disease Prediction", en, *SN Computer Science*, vol. 6, no. 4, p. 334, Apr. 2025, ISSN: 2661-8907. DOI: 10.1007/s42979-025-03860-2. [Online]. Available: https://doi.org/10.1007/s42979-025-03860-2.

[6] S. W. P. M. Janosi Andras et al., *Heart Disease*, UCI Machine Learning Repository, DOI: https://doi.org/10.24432/C52P4X, 1989.

[7] R. C. Detrano et al., "International application of a new probability algorithm for the diagnosis of coronary artery disease.", *The American journal of cardiology*, vol. 64 5, pp. 304–10, 1989. [Online]. Available: https://api.semanticscholar.org/CorpusID:23545303.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại đây.
2. **Hint:** Các file code gợi ý có thể được tải tại đây.
3. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại đây.
4. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây.
5. **Rubric:**

Mục	Kiến Thức	Đánh Giá
I.	- Hiểu bối cảnh bệnh tim mạch và tầm quan trọng của chẩn đoán sớm. - Nắm được thông tin về bộ dữ liệu Cleveland Heart Disease. - Biết các đặc trưng y tế chính (age, sex, cp, chol, trestbps, ...).	- Trình bày được lý do chọn bộ dữ liệu Cleveland. - Nhận biết và giải thích ý nghĩa các đặc trưng trong bộ dữ liệu. - Hiểu được mối liên hệ giữa dữ liệu y tế và mô hình học máy.
II.	- Kiến thức về xử lý đặc trưng (feature engineering). - Kiến thức về các mô hình cơ bản: Naive Bayes, KNN, Decision Tree. - Kiến thức về mô hình không giám sát K-means. - Kiến thức về Ensemble Learning (Stacking).	- Hiểu giả định của Naive Bayes và khi nào phù hợp. - Nhận biết vai trò của K trong KNN và độ sâu trong Decision Tree. - Giải thích nguyên lý K-means và lý do chọn $k = 2$. - Trình bày lợi ích của Ensemble trong việc kết hợp nhiều mô hình.
III.	- Kiến thức về triển khai code với scikit-learn. - Kỹ năng sử dụng cross-validation để tìm tham số tối ưu. - Biết cách đánh giá mô hình bằng accuracy, classification report, biểu đồ.	- Viết và hiểu được các hàm `evaluate_val`, `evaluate_test`. - Hiểu mục đích của StratifiedKFold trong tìm K tối ưu. - Giải thích được ý nghĩa kết quả accuracy và các chỉ số Precision, Recall, F1. - Biết so sánh hiệu quả giữa tập dữ liệu gốc và tập đã xử lý đặc trưng.